International Journal of Computer Vision and Signal Processing

LightBanglaHC Audit: Evaluation Validity, Inference Reconstruction, and Accounting Boundaries in Harmful-Content Classification

Abstract

We audit a stacked TF-IDF ensemble for harmful-content classification, using 47,817 labeled training rows and 11,955 test rows. The recorded weighted ensemble has pooled out-of-fold macro-F1 0.5314, compared with 0.5221 for its strongest base candidate. In a new paired logistic regression run, fitting TF-IDF on all training text yields 0.522061, versus 0.518872 with fold-fitted representations at the source iteration cap. LBFGS reduces this difference to 0.001750. The source-budget difference is about four times the recorded gain from outcome-derived ensemble weights. We also compare nonnested and nested meta training in a reduced three-base stack, obtaining 0.517144 and 0.515690 respectively with identical outer assessment features. Raw-string checks find 308 duplicate groups spanning folds and 52 groups with conflicting labels. A tiered taxonomy distinguishes label-free preprocessing, aggregate class statistics, supervised feature dependencies, stopping and selection. The profiled identity module is absent from the prediction path. Six base logistic regression fits imply 4,141,908 coefficients and intercepts; the exported value 0.183 is target-derived. Historical outputs, new executions and arithmetic are distinguished. Saved predictions and protocols support replication of this audit, rather than a claim of a new classifier or deployment advantage.

Keywords: Harmful-content classification, Evaluation validity, Stacked generalization, Accounting boundaries, Reproducibility.

© 2026, IJCVSP, CNSER. All Rights Reserved

IJCVSP

ISSN: 2186-1390 (Online)
http://cennser.org/IJCVSP

1. Introduction

A harmful content classifier's reported accuracy and efficiency are interpretable only when they refer to a specified prediction procedure. In sparse ensembles, predictions can depend on fitted representations, calibrated base estimators, and a second-stage head. Counting the head alone does not describe the system that produces its inputs [1].

We distinguish three objects: the *inference object*, which maps raw text to predictions; the *evaluation object*, which includes fitting, validation, and selection; and the *accounting object*, which defines the artifacts and computation included in a resource claim. Stacking [2] makes their relationship consequential: excluding a row from its own base fit does not necessarily isolate the full procedure when meta training features depend on meta validation labels. Global preprocessing and repeated selection create additional dependencies [3, 4, 5].

We examine one recorded implementation from the HerWILL AI for Digital Safety Datathon 2026. The supplied competition description is an operational source [6]. The updated notebook, retained outputs, competition description, and raw CSVs permit source inspection, data checks, inference reconstruction, and arithmetic accounting. Historical row-level OOF predictions and fitted artifacts remain unavailable. We execute new data checks, LR sensitivities and a reduced stacking comparison; these are distinct from replaying the original predictions. Documentation describes a mixed Bangla, English, and Banglish corpus; preprocessing alone would not establish this scope.

Our contributions are (1) a source-level reconstruction of the selected prediction and accounting paths, (2) full-data preprocessing and reduced-stacking comparisons with saved row-level probabilities, and (3) duplicate diagnostics and a tiered evaluation-exposure taxonomy. Section 3 defines the evidence categories. We introduce no new classifier.

2. Related Work

Hate speech, offensive language and implicit harm require documented annotation decisions [7]. Functional tests probe negation and counter speech [8]; harm and bias analysis requires stating whose interests are considered [9]. Prior Bengali hate-speech data [10] and BD-SHS [11] and BanglaBERT [12] provide Bangla context, not directly comparable scores or verified source datasets for this corpus. Data statements help document population and curation scope [13].

Stacked generalization learns from base predictions [2]. Breiman [14] studies cross-validated combinations of regres-

sion predictions; Ting and Witten [15] examine confidence features and second-level learners for classification. Super Learner studies cross-validated combination and risk minimization [16]; the source's squared-F1 heuristic is different. Hastie et al., Section 7.10.2 [17], explain why model assessment must include preparation and selection within permitted training data. Their supervised-selection example is not a numerical estimate of label-free TF-IDF bias here. Selection bias [3, 4] and leakage [18, 5] motivate tracing full feature ancestry. Calibration adds fitted state [19] and requires separate reliability diagnostics [20]. Overlapping folds complicate uncertainty [21]; paired tests require retained predictions and an appropriate design [22].

Green AI [1], energy reporting [23], reproducibility [24], and model cards [25] distinguish computation, replay and intended use. This study applies those established principles to one prediction and accounting record.

3. Study Design, Evidence, and Dataset Scope

3.1. Design and Audit Method

We trace fitting, prediction, selection, profiling and saving operations through their inputs; comments do not establish execution. We check CSV shapes, labels, IDs and duplication, and reconstruct metrics from the embedded confusion figure. These checks establish internal consistency, not authenticated historical execution.

Each claim has an evidence type: S (source observation), R (retained historical output), N (new execution), D (direct CSV check), A (arithmetic reconstruction), X (unverified external quantity), or P (prospective proposal). New experiments use Python 3.12.14, scikit-learn 1.8.0, NumPy 2.3.5, SciPy 1.17.0 and pandas 2.2.3. They do not recover the original fitted states. Supplements S1 and S2, included below, document version boundaries, protocols and artifact identity. The accompanying reproducibility package contains full execution records.

3.2. Available Artifacts and Version Boundaries

The canonical notebook has one code cell, 971 source lines, execution count one, two output streams, and one figure (S). Metadata records execution on 20 April 2026 for 9,142.89 seconds (R). Python metadata lists 3.12.12; historical package versions are absent. Output reports `Device: cuda`, while notebook metadata disables GPU. These retained fields are not independently authenticated execution records.

We use only the three-class 2026 archive. Historical variants are separated in Supplement S1. They do not support claims about this selected path.

3.3. Task and Checked Corpus

The supplied competition description maps 0 to Explicitly Toxic/Hateful, 1 to Subtly Toxic, and 2 to Non-toxic/Neutral. Training columns are `id`, `text`, and `y`; test columns are `id` and `text`. Shapes are (47,817, 3) and (11,955, 2) before processed columns are added (D), totaling 59,772 rows. Table 1 reports the class supports, which also match the updated notebook's printed counts.

Table 1: Directly checked supports (D); proportions and globally balanced class weights (A). The source computes weights before splitting (S).

Class	Support	Proportion (%)	Weight
Explicit	23,756	49.681	0.670946
Subtle	18,163	37.984	0.877553
Neutral	5,898	12.335	2.702442
Total	47,817	100.000	

The source uses $w_c = n/(3n_c)$ for LR, SVC, SGD, Meta-LR, and LightGBM sample weights. CNB has no explicit class weight argument. Since weights depend on every training label, they also depend on each fold subsequently used for assessment. The test rows have no labels and support no accuracy measurement.

Exact raw-string checks find 621 repeated training rows and 54 repeated test rows beyond first occurrences, and 254 distinct texts shared between train and test (D). New regenerated-fold counts are reported in Section 6.5. Identifiers are unique within each three-class file and disjoint between its train and test sets. Text overlap and identifier uniqueness are different properties. We assess a duplicate-group split and a raw cross-fold exclusion sensitivity. Shared-author, thread and semantic near-duplicate structure remain unexamined.

3.4. Documentation and Population Scope

The supplied description calls the corpus manually annotated Bangla, English and Banglish social-media content, emphasizing subtle toxicity, code mixing and noise. Language proportions and annotation agreement are not measured. Its details document lists CC BY-NC-ND 4.0, but supplies no full annotation manual, consent records or annotator-welfare documentation. Label names and script cues do not validate sarcasm detection or language identification. We describe the supplied documentation, not whether the original release had additional records.

4. Audited Prediction Pipeline

4.1. Cleaning, Signals, and Numerical Features

Let x be the original string after missing-value handling. The combined processed text is

$$s(x) = C(x) \,\|\, \texttt{SSEP} \,\|\, H(x), \tag{1}$$

where C cleans text and H produces signal tokens from the original input. The cleaner applies NFKC normalization, lowercasing, URL and mention substitutions, hashtag-marker removal, emoji substitution, repeated-letter reduction, repeated-punctuation markers, and whitespace com-

pression (S). Signals retain access to original capitalization and punctuation rather than only the cleaned string.

The lexical union has 21 Bangla, 23 transliterated and 29 English entries (73 distinct strings); the subtle inventory has 25 (S). Substring detection counts each distinct entry at most once. Signals repeat capped markers for lexical, sarcasm, threat, identity, script, capitalization, punctuation, length and neutral cues. The string "this is a salad" activates `sala`, a transliterated term used abusively in Bangla and Hindi; it also has benign and kinship uses. Here it is a substring inside "salad", not an abusive utterance. The match records a lexical condition rather than intent.

Twenty unscaled numerical features cover length, lexical diversity, punctuation, emoji, script, capitalization, lexical hits and pattern flags. They enter LightGBM only. Bangla regex runs divided by character length are not a fraction of Bangla characters. Cue names are code identifiers, not validated social constructs.

4.2. Four Globally Fitted Sparse Branches

Table 2 lists the feature branches and recorded widths. All use sublinear term frequency and are fitted on all processed training texts before the three-fold split. Test texts are transformed, not fitted on (S). Under the documented default interpretation of omitted smoothed-IDF and normalization options, a branch computes

$$\begin{aligned} u_{ij} &= (1+\ln t_{ij})\left[1+\ln\frac{n+1}{\mathrm{df}_j+1}\right], \quad t_{ij}>0, \\ \phi(x_i) &= u_i/\|u_i\|_2, \end{aligned} \tag{2}$$

with $u_{ij}=0$ otherwise. Historical package versions and fitted arrays are absent; current documentation is not evidence of the original runtime.[1]

CNB receives both character branches, totaling 130,000 columns. LR, SVC, and SGD receive the 230,105-column TF-IDF matrix. LightGBM receives that matrix plus 20 numerical columns. The global map ϕ exposes validation covariates to vocabulary construction, document-frequency filtering, feature caps, and IDF. A later estimator fit that excludes a row cannot undo this earlier representation fitting.

4.3. Base Candidates and Stacker

The source forms shuffled stratified three-fold splits with seed 42. Six base candidates are trained: two LR configurations, calibrated SVC, calibrated CNB, calibrated SGD, and LightGBM [26]. Table 3 gives explicit settings. The two LR candidates share an estimator family; six candidates do not imply six distinct algorithmic families.

Calibration adds fitted state to the probability path [19]. SVC and SGD use global class weights, whereas CNB does not. Their calibration fits remain inside each base fitting subset but share globally fitted features. No proper-score or reliability diagnostic evaluates these probabilities [20]. LightGBM's stopping criterion uses the same validation fold later scored for macro-F1; log loss and macro-F1 are different objectives.

4.4. OOF Features and Selected Test Predictor

Let V_1, V_2, V_3 partition $I=\{1,\dots,n\}$, and let $f(i)$ identify row i's fold. For base candidate b, let $h_b^{(-f)}$ be its probability-producing fit excluding V_f, with calibration or early stopping as specified. Training and test meta features differ:

$$\begin{aligned} z_i &= \left[h_b^{(-f(i))}(\phi(x_i))\right]_{b=1}^{6} \in \mathbb{R}^{18}, \\ \bar{z}(x) &= \left[\frac{1}{3}\sum_{f=1}^{3} h_b^{(-f)}(\phi(x))\right]_{b=1}^{6}. \end{aligned} \tag{3}$$

LightGBM additionally receives the numerical features, suppressed in this notation for readability. The notation excludes a fold from its base estimator fit, not from global preprocessing, class weights, or LightGBM stopping selection.

Three Meta-LR fits are trained from the single global OOF feature matrix using the same split scheme. Their test probabilities are averaged. The selected final predictor combines six averaged base outputs and the averaged Meta-LR output, so Meta-LR is an active component rather than merely an alternative candidate. If F_j is candidate j's pooled OOF macro-F1 and $p_j(x)$ its test probability vector, then

$$\alpha_j = \frac{F_j^2}{\sum_{k=1}^{7} F_k^2}, \qquad \hat{y}(x) = \arg\max_c \sum_{j=1}^{7} \alpha_j p_j(x)_c. \tag{4}$$

These weights use labeled development outcomes rather than a prespecified average. Figure 1 separates this prediction path from the independent profiling operation.

5. Recorded Results and Submission Provenance

5.1. Candidate and Ensemble Results

Table 4 preserves printed candidate scores and fold values. LR_C3 is the strongest base candidate. The weighted ensemble has pooled OOF macro-F1 0.5314, compared with 0.5306 for the simple average and 0.5221 for LR_C3 (R). The rounded weighted-versus-base difference is 0.0093, or 0.93 percentage points (A). It is descriptive; the source selects weights, stopping points, and candidates using evaluation information, and no untouched comparison or paired prediction file is supplied.

Pooled F1 is computed once from all OOF rows, not averaged over fold F1. OOF prediction uses a single held-out fit, whereas test prediction averages three fits at both levels. Assessing the latter requires a matched frozen procedure.

[1] Consulted API: https://scikit-learn.org/1.5/modules/generated/sklearn.feature_extraction.text.TfidfVectorizer.html.

Table 2: Source settings (S) and realized widths printed by the updated notebook (R). Numerical features are appended only for LightGBM.

Branch	Input	Analyzer	Grams / document frequency	Cap	Width
Word	$s(x)$	Word; non-whitespace tokens	1–3; min 1, max 0.98	100,000	100,000
Character	$s(x)$	`char_wb`	2–5; min 2	80,000	80,000
Clean character	$C(x)$	`char`	1–4; min 2	50,000	50,000
Signal	$H(x)$	Word; non-whitespace tokens	1–2; min 1	10,000	105
Combined TF-IDF				240,000	230,105
LightGBM input	TF-IDF + numerical		20 fixed numerical features		230,125

Table 3: Invoked settings in the updated source (S). Unspecified settings depend on the unrecorded historical library versions.

Candidate	Input	Explicit configuration
LR_C8, LR_C3	TF-IDF	$C = 8$ or 3; saga; multinomial; max. 2,000 iterations; tolerance 10^{-3}; global class weights; seed 42.
SVC	TF-IDF	LinearSVC, $C = 0.5$; max. 3,000 iterations; global class weights; seed 42; isotonic calibration, CV = 2.
CNB	Two character branches	$\alpha = 0.05$; `norm=True`; isotonic calibration, CV = 2; no explicit class weights.
SGD	TF-IDF	Modified Huber; $\alpha = 10^{-4}$; max. 200 iterations; global class weights; seed 42; isotonic calibration, CV = 2.
LightGBM	TF-IDF + 20 numerical	Multiclass log loss; 63 leaves; rate 0.1; feature fraction 0.7; bagging fraction 0.8, frequency 5; min. child samples 20; $L_1 = 0.1$, $L_2 = 0.5$; `is_unbalance=True`; global sample weights; seed 42; max. 1,000 rounds; early-stopping patience 80.
Meta-LR	18 probabilities	$C = 10$; max. 500 iterations; multinomial; global class weights; seed 42; solver not overridden.

Each threshold search evaluates $30 \times 30 = 900$ pairs, adjusting classes 0 and 1 while leaving class 2's divisor at one (S). The interval 0.1–0.9 omits $(1, 1)$, so its best score can be below the unadjusted result. The selected predictor uses the unadjusted weighted ensemble; displayed thresholds 0.90, 0.87 and 1.00 are not on its path.

5.2. Confusion Counts and Error Structure

Table 5 transcribes the nine integer annotations in the updated notebook's embedded confusion figure. Row sums match the raw CSV supports and total 47,817. Reconstruction yields macro-F1 0.5314177985 and accuracy 0.5803166238 (A), consistent with the printed 0.5314. The three classwise values agree with the figure's four-decimal labels.

Neutral has the lowest F1 and recall. Of 4,829 examples predicted Neutral, 2,583 have a toxic label; 3,652 actual Neutral examples are assigned a toxic label. The largest off-diagonal count is 7,502 Subtle examples predicted Explicit. Without paired row-level outputs, causes and joint candidate errors remain unknown. An always-Explicit predictor has arithmetic macro-F1 $2n_0/[3(n + n_0)] = 0.221275$ (A), not a measured baseline experiment.

5.3. Test Export and Platform Claims

The updated notebook prints 6,414 Explicit, 4,401 Subtle, and 1,140 Neutral test predictions (R). A newly supplied CSV contains exactly these class counts, 11,955 rows, columns `id,y_pred,parameters`, IDs matching the three-class test file in order, and constant `parameters`=0.183 (D). This establishes aggregate and schema consistency with the recorded export. Without fitted states or row-level historical probabilities, it does not authenticate the origin of every prediction.

The source writes additional files with declared NOP values 0.1, 0.3, and 1. Number of parameters (NOP) means a count under a stated model boundary, not a freely chosen denominator. A zero-valued loop entry is skipped because it equals the initial profiled value; a later summary recommends that filename despite no matching save. These source operations do not establish platform acceptance. No platform scores are reported without a scoring receipt.

5.4. Timing and Retained State

The recorded LR training sections take 2,601.7 and 2,625.0 seconds, and LightGBM takes 3,802.8 seconds (R). These three sections alone total 9,029.5 seconds, about 150.5 minutes (A), before other work. The notebook metadata reports about 152.4 minutes. Thus its concluding "25–35 minutes" claim is inconsistent with its retained timings. Those timings are historical output, not a newly controlled runtime benchmark, and no inference latency measurement is supplied.

LightGBM reports best iterations 149, 173, and 182 (R). The inspected source serializes no fitted vectorizers, base models, boosters, meta estimators, or historical OOF arrays. In-memory test arrays are generated before export, but the absence of saved model files prevents fresh-process raw-text replay. Historical environment uncertainty remains even

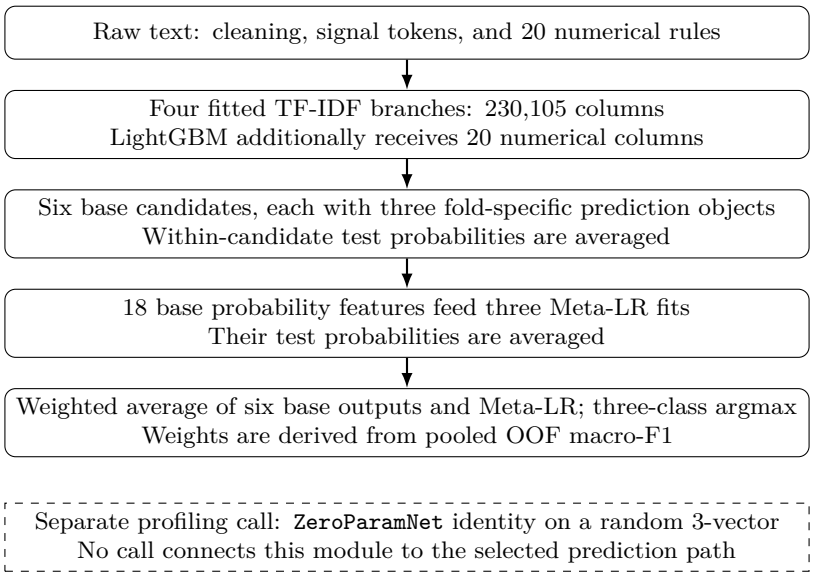


Figure 1: Source-derived computation (S), not a measured deployment benchmark. Base outputs also enter the final weighted average directly, as specified in Equation 4.

Table 4: Recorded pooled and fold macro-F1 (R). Ensemble variants are scored on the same pooled OOF rows; no uncertainty interval is inferred from dependent folds.

Candidate	Role	Pooled	Fold 1	Fold 2	Fold 3
LR_C8	Base	0.5169	0.5158	0.5122	0.5227
LR_C3	Base	0.5221	0.5209	0.5171	0.5281
SVC	Base	0.4824	0.4860	0.4827	0.4782
CNB	Base	0.4391	0.4403	0.4356	0.4414
SGD	Base	0.4691	0.4675	0.4719	0.4678
LightGBM	Base	0.5160	0.5154	0.5136	0.5191
Meta-LR	Stacker / ensemble member	0.5218	0.5183	0.5169	0.5299
Weighted	Selected ensemble	0.5314	–	–	–
Simple average	Alternative	0.5306	–	–	–
Weighted + thresholds	Alternative	0.5300	–	–	–
Average + thresholds	Alternative	0.5287	–	–	–

though the source and recorded dimensions can now be inspected.

6. Evaluation Validity

6.1. Different Types of Evaluation Exposure

The retained score describes development predictions. Table 6 separates mechanisms by the information they use and their structural reach. The tiers prioritize validation work; they are not measured effect sizes or a universal ordering of harm.

The TF-IDF fits precede fold construction, but test text is only transformed. Thus this is intra-training covariate exposure, not fitting on the unlabeled test set. Global class weights use $w_{I,c} = |I|/(3n_{I,c})$. Regenerated fold-local weights differ from these global values by at most 0.005506% relatively (A), reflecting the close stratified class counts. This small statistic difference should not be presented as equivalent in scope to the meta learner's full supervised ancestry. It does not by itself prove a negligible prediction effect.

6.2. Nonnested Meta-Training Dependencies

For a meta learner $q^{(-a)}$ trained on $\{(z_i, y_i) : i \notin V_a\}$ and evaluated on V_a, take a meta-training row $i \in V_b$, $b \neq a$. Its features came from base fits on $I \setminus V_b$, a set containing V_a. Thus

$$\begin{aligned} y_{V_a} &\longrightarrow h_m^{(-b)} \longrightarrow z_i \\ &\longrightarrow \text{meta-training data} \longrightarrow q^{(-a)}. \end{aligned} \tag{5}$$

A row in V_a does receive base features from an estimator fit excluding V_a. However, the meta model evaluating that row was trained on other features whose construction used y_{V_a}. Row-level OOF exclusion therefore does not isolate the complete stacker. This structural dependency remains even if global preprocessing and class weights are provisionally accepted.

Table 5: Selected weighted-ensemble OOF confusion counts (R), with classwise metrics reconstructed from the displayed counts (A). Rows are actual labels; columns predicted labels.

Actual	Explicit	Subtle	Neutral	Support	Precision	Recall	F1
Explicit	16,464	6,331	961	23,756	0.6501	0.6930	0.6709
Subtle	7,502	9,039	1,622	18,163	0.5118	0.4977	0.5046
Neutral	1,360	2,292	2,246	5,898	0.4651	0.3808	0.4188
Predicted	25,326	17,662	4,829	47,817	Macro-F1 0.531418 (A)		

Table 6: Structural exposure taxonomy (S/A/P). Magnitude is unmeasured unless an observed diagnostic is stated; an observed score gap is not a bias bound.

Mechanism / tier	Information and reach	Interpretation and required control
TF-IDF / covariate exposure	Label-free validation text enters vocabulary, frequency filters, caps and IDF.	Transductive representation exposure. Fit each branch on the permitted training subset; compare with the global arm. No label-path claim.
Class weights / aggregate exposure	Three class counts include validation labels; fitting sample weights inherit them.	Low-dimensional label statistics. Here stratification makes fold-local values differ by at most 0.005506% relatively (A); measure any prediction effect separately.
Early stopping / supervised selection	LightGBM assessment labels select boosting rounds through log loss.	Reuse of assessment outcomes. Use a stopping partition inside the training subset, then assess the frozen fit.
Meta-stacking / supervised feature ancestry	Meta-training features are generated by base fits containing meta-assessment labels.	Full supervised dependency path. Rebuild inner OOF features within every outer training subset.
Ensemble weights / aggregate selection	Seven pooled F1 scores determine normalized squared weights.	Near-uniform weights; observed gain 0.0008. Estimate optimism only with protected assessment labels.
Threshold and candidate choice / repeated selection	Two grids of 900 choices and four final variants use the same labeled OOF rows.	Multiple adaptive comparisons. Freeze decisions within training-only selection partitions.

Rebuilding base OOF features only once for all meta folds preserves this path. It must instead be regenerated within the training partition of each outer assessment split, and again within the allowed fitting subset of each selection split. Equation 5 establishes missing isolation, not the magnitude or direction of score distortion. The reduced comparison below executes both meta-training constructions with the same outer assessment probabilities.

6.3. Stopping, Near-Uniform Weights, and Selection

LightGBM uses its assessment labels for stopping and later reports macro-F1 on the same rows. Reporting a different objective does not create a new assessment sample. Ensemble weights are functions of the pooled OOF F1 values, so each evaluation label can also affect the aggregation rule.

Using the seven printed four-decimal candidate scores in Equation 4 gives approximate weights 0.154991, 0.158125, 0.134992, 0.111846, 0.127651, 0.154452, and 0.157943 for LR_C8, LR_C3, SVC, CNB, SGD, LightGBM, and Meta-LR respectively (A). Uniform weight is $1/7 = 0.142857$; the largest absolute departure is about 0.031011. The source used unrounded scores, so these are reconstructed approximations, not recovered fitted weights.

The recorded weighted score 0.5314 exceeds the simple average 0.5306 by 0.0008, or 0.08 percentage points (A). This is a small observed development-set gain. It is *not* an upper bound on leakage or selection optimism: both probability arrays share other evaluation dependencies, and no untouched paired comparison is available. We therefore give this mechanism narrower emphasis than meta-feature ancestry.

Each threshold grid assesses 900 choices on the same OOF labels. Their selected results and unadjusted alternatives are then compared to choose one of four final variants. These are selection opportunities [3, 4]; no measured bias is assigned to them or to the rounded 0.0093 ensemble-versus-base difference.

6.4. Dependence and Uncertainty

The three fitting subsets overlap: each pair shares roughly one-third of the full corpus, or half of either fitting subset. They are not independent repetitions. Stratification addresses class balance but does not protect against duplicates, shared authors, threads, or temporal dependence. Seed-42 fold indices are now regenerated from the supplied CSV using scikit-learn 1.8.0. They are saved with IDs and

labels. They are a new reconstruction, not an authenticated historical fold file; their duplicate counts appear below.

There is no universally unbiased K-fold variance estimator based only on observed fold errors [21]. Paired comparisons also require retained row level predictions and a procedure appropriate to the data's independent units [22]. We bootstrap paired fixed OOF predictions below. This conditions on the fitted models, omits training uncertainty, and supports no claim of general predictive superiority.

6.5. New Checks and Controlled Comparisons

Table 7 separates new executions from historical scores. Both SAGA LR arms use the full CSV, unchanged row order, three shuffled stratified folds with seed 42, $C = 3$, tolerance 10^{-3} and 2,000 iterations. Global class weights are fixed to isolate TF-IDF fit scope. Global fitting gives 0.522061 versus 0.518872 for training-fold fitting (N), a difference of 0.003189 (A). This is about four times the historical weighted-versus-uniform gain of 0.0008 and one-third of the ensemble-versus-base difference of 0.0093. These comparisons involve different prediction objects and are descriptive, not a decomposition of ensemble bias.

All six initial LR fits reach their iteration cap. We therefore also run LBFGS with the same $C = 3$ multinomial L2 objective, tolerance 10^{-6} and maximum 2,000 iterations. Its convergence records and objective-gradient diagnostics are in Supplement S2. Both LBFGS arms stop below the iteration cap, with recorded gradients below 10^{-6}. Their global-minus-fold difference is 0.001750, smaller than the source-budget difference. The preprocessing sensitivity therefore depends on optimization as well as fit scope. This does not authenticate the historical environment. The paired bootstrap uses 5,000 class-stratified row resamples and 2,000 exact-raw-text cluster resamples, both with seed 42. It rescores fixed predictions; it neither retrains models nor resolves overlapping training folds, shared users or threads.

For reduced stacking, every base feature fit and class-weight calculation uses its permitted training subset. Calibrated SGD uses modified Huber loss, $\alpha = 10^{-4}$ and 200 iterations; calibrated CNB uses the two character branches, $\alpha = 0.05$ and normalization. Both use two-fold isotonic calibration with ensemble retention. Meta-LR has nine probability inputs, $C = 10$, 500 iterations and outer-training class weights. Nonnested meta training uses the shared outer-OOF matrix. Nested training rebuilds three-fold base OOF features inside each outer training subset. Both arms are assessed on exactly the same base probabilities, generated by one fit on all outer training rows. No LightGBM, threshold search, outcome-derived ensemble weights or candidate selection enters this contrast. All 12 reduced-stack LR fits reach the 2,000-iteration cap; the comparison uses the source training budget. This tests a reduced procedure, not the recorded six-candidate weighted ensemble.

The raw-text check finds 461 repeated groups containing 1,082 rows; 308 groups and 771 rows (1.6124% of training rows) cross the regenerated folds. Conflicts occur in 52 groups and 130 rows (0.2719%); 36 conflict groups also span folds (N). Applying the notebook's cleaner $C(x)$ yields 495 repeated groups, 336 spanning groups and 67 conflict groups. The spanning cleaner groups contain 880 rows (1.8403%); conflicts affect 200 rows (0.4183%). Cleaner-group folds have zero spanning groups. Their LR score and the exclusion sensitivity in Table 7 change the sampling policy and, for exclusion, the corpus; these are not paired estimates of duplication alone. Exclusion preserves original fold IDs and leaves zero spanning raw-text groups; only the group-split arm guarantees isolation under $C(x)$. The complete notebook has not been replayed. New scripts and probabilities permit replication of the narrower comparisons actually reported.

7. Parameter Boundaries and Score Provenance

7.1. The Identity Module Is a Different Object

The source defines `ZeroParamNet.forward(x)` to return x unchanged. It has no trainable parameters. Profiling, when available, calls this module on a random three-element vector after the selected predictions already exist (S). No invocation applies the module to raw text, base probabilities, meta features, or selected outputs. Its zero count therefore describes a separate identity operation, not the harmful-content predictor.

The retained output prints `Manual NOP: 0` and `Final thop NOP: 0`, without the success line printed by the successful profiling branch. The variable is initialized to zero and remains zero when THOP is unavailable or fails. Consequently, the final printed zero alone cannot establish successful profiling. The source uses one positional input to this separate module, whereas the supplied competition description illustrates a two-input model call with token IDs and an attention mask. These interfaces do not determine a whole-system accounting boundary.

7.2. Structural Inventory of the Selected Predictor

The selected path needs six three-fold base candidates, three Meta-LR fits, seven weights, four fitted vocabularies/IDF arrays and fixed text/numerical rules. No learned PyTorch head is invoked.

At recorded TF-IDF width $D = 230{,}105$, each three-class multinomial LR fit has $3D$ coefficients and three intercepts. The two LR candidates each require three fits for their stored test averages. Meta-LR has 18 input probabilities and three output scores:

$$\begin{aligned} N_{\text{one base LR}} &= 3D + 3 = 690{,}318, \\ N_{\text{six base LR}} &= 6(3D + 3) = 4{,}141{,}908, \\ N_{\text{three Meta LR}} &= 3(3 \times 18 + 3) = 171. \end{aligned} \tag{6}$$

These are arithmetic scalar inventories for specified matrices (A), not measured memory occupancy or a complete contest NOP. Coefficients for SVC and SGD, CNB state,

Table 7: New full-data comparisons (N). All F1 values are pooled macro-F1. Differences are observed protocol sensitivities, not universal leakage bounds.

Comparison	Macro-F1 / difference	Control and qualification
SAGA LR, global / fold-fitted TF-IDF	0.522061 / 0.518872	Same folds; global class weights fixed; all six fits hit 2,000 iterations.
Global minus fold-fitted LR	0.003189	Fixed-prediction stratified bootstrap 95% interval [0.000808, 0.005643]; raw-text cluster interval [0.000802, 0.005558].
LBFGS LR, global / fold-fitted TF-IDF	0.523143 / 0.521393	Same penalized objective; different solver; tolerance 10^{-6}.
Reduced stack, nonnested / nested	0.517144 / 0.515690	LR_C3, calibrated SGD and CNB, Meta-LR; identical outer assessment features.
Nonnested minus nested reduced stack	0.001454	Local preprocessing and class weights in both arms; source-budget SAGA fits.
LBFGS LR, cleaner-group split	0.519044	No cleaner-normalized group crosses folds; local weights and TF-IDF.
LBFGS LR, raw cross-fold rows excluded	0.518443	Remove all 771 spanning raw-text rows; retain original folds on 47,046 rows.

calibrators, LightGBM trees, vocabularies, and IDF arrays remain outside this subtotal. Internal calibration can retain multiple fitted classifiers/functions per outer object; historical defaults and fitted internals are unknown [27].

No complete serialized artifact, peak RAM measurement, raw-text inference latency, or energy benchmark is available. Scalar counts, fitted state, bytes, RAM, latency and energy are distinct [23]. Comparisons need the same boundary and models that generate probabilities for unseen text.

7.3. Fractional Export Values Are Not Measurements

The source sets the main export field by rounding $F/2.9$ to three decimal places (S), producing 0.183 (R/D). This is a target-derived denominator, not a structural parameter measurement. The assigned values 0.183, 0.1, and 0.3 are fractional, whereas a count of scalar parameters is an integer. The separate identity operation has zero trainable parameters; the selected fitted ensemble has additional state. These observations describe implementation and measurement provenance, not a verified organizer ruling or a claim about author intent.

With $F = 0.5314177984731835$ and $\epsilon = 5 \times 10^{-16}$, the local ratio $F/(N+\epsilon)$ gives about 1.0628×10^{15} at $N = 0$ and 2.9039 at $N = 0.183$ (A). These are arithmetic projections, not verified leaderboard scores.

7.4. Authoritative Scoring Remains Unresolved

The supplied details document gives $0.4F + 0.6/[\log(N + 1) + \epsilon]$ without a log base, but later quotes $F/(N + \epsilon)$. The original formula image, official scoring code and classical-model count boundary are not retained. The notebook's local ratio is not verification of the contest contract. For the logarithmic expression, assuming natural logarithms and $N = 0.183$ gives an efficiency term approximately 3.57029 (A); a small fractional denominator can therefore dominate that hypothetical score. Neither numerical plausibility nor a reconstructed score establishes the organizer's rule. We report no hidden-set F1 or leaderboard result. Supplement S1 preserves version provenance without treating unverified platform reports as measurements.

8. Corrected Prospective Protocol

Freeze data bytes, label meanings, sampling units and permitted uses. Inspect duplicate/conflict groups and document label-conflict treatment. Keep row-stratified and group-aware assessments separate. Within each outer training subset, rebuild inner base OOF features, fit preprocessing and class weights locally, and keep calibration and stopping inside training data. Prespecify the assessment predictor: our reduced stack uses one outer-training base fit in both arms; a fold-averaged predictor requires a matched evaluation.

For the complete weighted ensemble, reserve combination weights, thresholds and candidate choice for training-only selection. The reduced stack does not validate these additional steps. Preserve IDs, folds, probabilities, versions, optimizer diagnostics and fitted objects. Reload the complete raw-text predictor and check probability equivalence before measuring bytes, peak RAM, latency or energy [24]. Define contest NOP from the original rules. Supplement S2 identifies executable scripts and retained outputs.

9. Limitations and Responsible Interpretation

The updated source and retained outputs are inspectable, and the supplied CSVs agree with its printed supports. However, no hash recorded during the historical run establishes that those exact bytes were used. The audit regenerates folds, checks a new export, and executes LR and reduced-stacking comparisons. It does not recover historical OOF rows, inspect historical fitted states, or verify platform acceptance. Unknown package

versions and contradictory device metadata limit replay. Exact-string duplication, conflicting labels and regenerated cross-fold allocation are checked. Semantic duplicates and shared-user/thread structure remain unexamined.

The threats concern fitting and selection exposure, one-record generalization, incomplete annotation documentation, dependent observations and mismatched accounting boundaries. Controlled differences quantify specific protocol changes, not total historical optimism or deployment cost. Three-way argmax is not a moderation policy: substring and identity cues do not establish endorsement, quotation or reclaimed usage [9, 8]. No deployment safety, appeal process or resource advantage is measured. Public-source collection does not itself establish consent or redistribution permission.

10. Conclusion

This case study reconstructs the updated ensemble record: 47,817 training rows, four fitted TF-IDF branches, six base candidates, a Meta-LR component, and a selected weighted ensemble. Its reconstructed macro-F1 0.531418 agrees with the notebook's 0.5314; the rounded comparison with LR_C3 at 0.5221 remains descriptive. Regenerated folds reveal repeated and conflicting text groups; the new CSV matches recorded export structure and class totals. Historical versions are documented in Supplement S1. Global TF-IDF raises macro-F1 by 0.003189 at the source SAGA budget; the converged LBFGS sensitivity gives a smaller difference, 0.001750. The reduced stack comparison gives 0.517144 for nonnested and 0.515690 for nested meta training, with the same outer assessment features. Both results make the evaluation protocol part of the reported outcome; neither measures the bias of every historical selection step.

Row-level OOF exclusion does not isolate the full procedure when assessment labels remain ancestors of meta-training features or selection. A zero-parameter identity operation omits the fitted predictor that supplies its inputs, and a target-derived fractional denominator is not a parameter measurement. Predictive and efficiency claims need a specified inference object, fitting ancestry, accounting boundary and evidence type.

Data, Code and Declarations.

The accompanying reproducibility package contains the inspected source, fold indices, executable scripts, new OOF probabilities, results and a SHA-256 ledger. Supplements S1 and S2 below summarize the included materials. Dataset access and any redistribution remain subject to the original competition terms; the supplied description lists CC BY-NC-ND 4.0, which is not independently authenticated here. The available audit record contains no ethics approval or exemption identifier, consent record, or separate redistribution authorization. We do not infer these from publicly accessible source posts. No funding was received, and we declare no competing interests. The earlier TinyLinear manuscript was an internal draft, never submitted or published. All authors approve this audit. AI assistance was used for source inspection, literature organization, calculations, execution of the new experiments, drafting and typesetting. The reported new executions are not an independent author replication.

Appendix A. Supplement S1: Version Boundaries

The main analysis concerns only the updated three-class notebook. An earlier TinyLinear draft reported 254,983 features, five folds and a 39-parameter linear head, with pooled macro-F1 0.5295. Its test average required five heads (195 head parameters); five three-class LR fits at that feature width imply $5[3(254{,}983) + 3] = 3{,}824{,}760$ coefficients and intercepts (A). Those are different artifacts from the updated six-candidate ensemble and are not combined with its 0.5314 result. The four-class files belong to a separate task version. An earlier NOP-39 CSV and the current NOP-0.183 export are retained as distinct files. Neither authenticates a scoring receipt. The prior internal draft has no simultaneous submission or publication status.

Appendix B. Supplement S2: Execution and Artifact Ledger

The included scripts implement paired LR, reduced stacking, duplicate/bootstrap, convergence and exclusion checks. They preserve IDs, labels, seed-42 folds, class order and software versions. All probability files are independently rescored and checked against the supplied CSV before packaging. Raw and normalized duplicate group counts, exclusion masks, group-fold IDs and conditional bootstrap intervals are saved. Optimizer iterations and cap indicators are retained rather than suppressed.

The data ledger identifies the training CSV by SHA-256 prefix `a269ad841f824c6e`; the updated notebook by `1dcbb4150a296155`; and extracted notebook source by `b1bf6978a045cb0a`. Full digests, rather than these display prefixes, are in the attached ledger. The initial global and fold-fitted LR prediction files have prefixes `78dd2758341b83f9` and `1f508fb4b7ce489b`. These identify the analyzed artifacts, not bytes authenticated during the historical competition run. The full companion supplement supplies detailed settings, convergence diagnostics, bootstrap construction and the complete ledger. Independent author execution is separate from these assistant-executed comparisons and is not claimed.

References

[1] R. Schwartz, J. Dodge, N. A. Smith, O. Etzioni, Green AI, Communications of the ACM 63 (12) (2020) 54–63. doi:10.1145/3381831.

[2] D. H. Wolpert, Stacked generalization, Neural Networks 5 (2) (1992) 241–259. doi:10.1016/S0893-6080(05)80023-1.

[3] S. Varma, R. Simon, Bias in error estimation when using cross-validation for model selection, BMC Bioinformatics 7 (1) (2006) 91. doi:10.1186/1471-2105-7-91.

[4] G. C. Cawley, N. L. C. Talbot, On over-fitting in model selection and subsequent selection bias in performance evaluation, Journal of Machine Learning Research 11 (70) (2010) 2079–2107.
URL https://jmlr.org/papers/v11/cawley10a.html

[5] S. Kapoor, A. Narayanan, Leakage and the reproducibility crisis in machine-learning-based science, Patterns 4 (9) (2023) 100804. doi:10.1016/j.patter.2023.100804.

[6] HerWILL, HerWILL: AI for Digital Safety Datathon 2026: supplied competition description and dataset materials, competition overview, dataset description and evaluation instructions; operational documentation retained with the audit artifacts (2026).

[7] T. Davidson, D. Warmsley, M. Macy, I. Weber, Automated hate speech detection and the problem of offensive language, Proceedings of the International AAAI Conference on Web and Social Media 11 (1) (2017) 512–515. doi:10.1609/icwsm.v11i1.14955.

[8] P. Röttger, B. Vidgen, D. Nguyen, Z. Waseem, H. Margetts, J. Pierrehumbert, HateCheck: Functional tests for hate speech detection models, in: C. Zong, F. Xia, W. Li, R. Navigli (Eds.), Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics and the 11th International Joint Conference on Natural Language Processing (Volume 1: Long Papers), Association for Computational Linguistics, Online, 2021, pp. 41–58. doi:10.18653/v1/2021.acl-long.4.

[9] S. L. Blodgett, S. Barocas, H. Daumé III, H. Wallach, Language (technology) is power: A critical survey of "bias" in NLP, in: D. Jurafsky, J. Chai, N. Schluter, J. Tetreault (Eds.), Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics, Association for Computational Linguistics, Online, 2020, pp. 5454–5476. doi:10.18653/v1/2020.acl-main.485.

[10] N. Romim, M. Ahmed, H. Talukder, M. S. Islam, Hate speech detection in the bengali language: A dataset and its baseline evaluation, arXiv:2012.09686 (2020). doi:10.48550/arXiv.2012.09686.

[11] N. Romim, M. Ahmed, M. S. Islam, A. Sen Sharma, H. Talukder, M. R. Amin, BD-SHS: A benchmark dataset for learning to detect online Bangla hate speech in different social contexts, in: N. Calzolari, F. Béchet, P. Blache, K. Choukri, C. Cieri, T. Declerck, S. Goggi, H. Isahara, B. Maegaard, J. Mariani, H. Mazo, J. Odijk, S. Piperidis (Eds.), Proceedings of the Thirteenth Language Resources and Evaluation Conference, European Language Resources Association, Marseille, France, 2022, pp. 5153–5162.
URL https://aclanthology.org/2022.lrec-1.552/

[12] A. Bhattacharjee, T. Hasan, W. Ahmad, K. S. Mubasshir, M. S. Islam, A. Iqbal, M. S. Rahman, R. Shahriyar, BanglaBERT: Language model pretraining and benchmarks for low-resource language understanding evaluation in Bangla, in: M. Carpuat, M.-C. de Marneffe, I. V. Meza Ruiz (Eds.), Findings of the Association for Computational Linguistics: NAACL 2022, Association for Computational Linguistics, Seattle, United States, 2022, pp. 1318–1327. doi:10.18653/v1/2022.findings-naacl.98.

[13] E. M. Bender, B. Friedman, Data statements for natural language processing: Toward mitigating system bias and enabling better science, Transactions of the Association for Computational Linguistics 6 (2018) 587–604. doi:10.1162/tacl_a_00041.

[14] L. Breiman, Stacked regressions, Machine Learning 24 (1996) 49–64. doi:10.1023/A:1018046112532.

[15] K. M. Ting, I. H. Witten, Issues in stacked generalization, Journal of Artificial Intelligence Research 10 (1999) 271–289. doi:10.1613/jair.594.

[16] M. J. van der Laan, E. C. Polley, A. E. Hubbard, Super learner, Statistical Applications in Genetics and Molecular Biology 6 (1), article 25. doi:10.2202/1544-6115.1309.

[17] T. Hastie, R. Tibshirani, J. Friedman, The Elements of Statistical Learning: Data Mining, Inference, and Prediction, 2nd Edition, Springer, New York, 2009, section 7.10.2. doi:10.1007/978-0-387-84858-7.

[18] S. Kaufman, S. Rosset, C. Perlich, O. Stitelman, Leakage in data mining: Formulation, detection, and avoidance, ACM Transactions on Knowledge Discovery from Data 6 (4) (2012) 15:1–15:21. doi:10.1145/2382577.2382579.

[19] A. Niculescu-Mizil, R. Caruana, Predicting good probabilities with supervised learning, in: Proceedings of the 22nd International Conference on Machine Learning, ICML '05, Association for Computing Machinery, 2005, pp. 625–632. doi:10.1145/1102351.1102430.

[20] J. Vaicenavicius, D. Widmann, C. Andersson, F. Lindsten, J. Roll, T. Schön, Evaluating model calibration in classification, in: K. Chaudhuri, M. Sugiyama (Eds.), Proceedings of the Twenty-Second International Conference on Artificial Intelligence and Statistics, Vol. 89 of Proceedings of Machine Learning Research, PMLR, 2019, pp. 3459–3467.
URL https://proceedings.mlr.press/v89/vaicenavicius19a.html

[21] Y. Bengio, Y. Grandvalet, No unbiased estimator of the variance of K-fold cross-validation, Journal of Machine Learning Research 5 (2004) 1089–1105.
URL https://www.jmlr.org/papers/v5/grandvalet04a.html

[22] R. Dror, G. Baumer, S. Shlomov, R. Reichart, The hitchhiker's guide to testing statistical significance in natural language processing, in: I. Gurevych, Y. Miyao (Eds.), Proceedings of the 56th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), Association for Computational Linguistics, Melbourne, Australia, 2018, pp. 1383–1392. doi:10.18653/v1/P18-1128.

[23] P. Henderson, J. Hu, J. Romoff, E. Brunskill, D. Jurafsky, J. Pineau, Towards the systematic reporting of the energy and carbon footprints of machine learning, Journal of Machine Learning Research 21 (248) (2020) 1–43.
URL https://jmlr.org/papers/v21/20-312.html

[24] J. Pineau, P. Vincent-Lamarre, K. Sinha, V. Larivière, A. Beygelzimer, F. d'Alché Buc, E. Fox, H. Larochelle, Improving reproducibility in machine learning research (A report from the NeurIPS 2019 reproducibility program), Journal of Machine Learning Research 22 (164) (2021) 1–20.
URL https://jmlr.org/papers/v22/20-303.html

[25] M. Mitchell, S. Wu, A. Zaldivar, P. Barnes, L. Vasserman, B. Hutchinson, E. Spitzer, I. D. Raji, T. Gebru, Model cards for model reporting, in: Proceedings of the Conference on Fairness, Accountability, and Transparency, Association for Computing Machinery, 2019, pp. 220–229. doi:10.1145/3287560.3287596.

[26] G. Ke, Q. Meng, T. Finley, T. Wang, W. Chen, W. Ma, Q. Ye, T.-Y. Liu, LightGBM: A highly efficient gradient boosting decision tree, in: Advances in Neural Information Processing Systems, Vol. 30, 2017.
URL https://papers.neurips.cc/paper_files/paper/2017/hash/6449f44a102fde848669bdd9eb6b76fa-Abstract.html

[27] F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, M. Blondel, P. Prettenhofer, R. Weiss, V. Dubourg, J. Vanderplas, A. Passos, D. Cournapeau, M. Brucher, M. Perrot, É. Duchesnay, Scikit-learn: Machine learning in Python, Journal of Machine Learning Research 12 (85) (2011) 2825–2830.
URL https://jmlr.org/papers/v12/pedregosa11a.html